

1) What is Python?

Python is a high-level, interpreted programming language known for its simplicity and readability. It allows developers to write clear programs for both small and large-scale projects. Python is used for.

- web development (e.g. Django, Flask)
- Data science and machine learning (e.g. Pandas, TensorFlow)
- Automation (e.g.) scripting tasks)
- Software development and more.

2) Why is Python Popular?

- Easy to learn: Python's syntax is simple and very close to natural language, making it a great language for beginners.
- community support: It has a meaning you'll find lots of tutorials, resources, and libraries.
- cross-platform: Python works on different operating systems (Windows, macOS, Linux, etc)
- versatile: whether it's web development, data analysis or automation, Python has libraries for almost everything.

3. Setting up Python Environment

Step 1: Download Python

- Visit: Python's official website
- Download the latest version of Python for your operating system (Windows, macOS, or Linux)

• Installation on Windows:

- Check the option "Add Python to PATH" during installation.
- Choose "Install Now" or customize installation of

Step 2: Installing IDE (Integrated Development Environment)

- IDE options: Python can be written in any text editor, but for ease, it's better to use an IDE, some popular ones are:
- Pycharm: A full-featured IDE (download from [here](#))
- VScode: A lightweight editor with python support (download from [here](#)) - Recommended
- Jupyter notebook: Great for data science and learning python interactively (install with pip `install jupyter notebook`)

Step 3: Verify Installation

- Open the command prompt or terminal.
- Type `python --version` or `python3 --version` to verify that python is successfully installed.

4. Writing your First Python Program

Let's write a simple program to understand how python works.

Step 1: Open a Text Editor or IDE

- Open any text editor like notepad or an IDE like Pycharm / VScode.

Step 2: Write your First Python code

```
Print ("Hello, world!")
```

This code will print "Hello world!" on the screen

Step 3: Run the Program

- on IDE: Click the "Run" button
- on Terminal: save the file as `hello.py` and navigate to the file location in the terminal. Then run:

```
python hello.py
```

5. Python as an Interpreted Language
unlike other compiled language like C or Java, Python executes the code line by line, which makes debugging easy. Python doesn't require you to compile your code into machine language; Python interpreter takes care of it.

Benefits of Interpreted Language:

- Easier debugging: Errors are reported line by line
- Faster development: You can directly run the code without worrying about compiling

6. Key Features of Python

- 1) Simple Syntax: Easy to read and write similar to English.
 - 2) Interpreted: Python is executed line by line.
 - 3) Dynamically Typed: No need to declare variable type explicitly.
 - 4) Object-oriented: Supports OOP (Object-oriented Programming) like classes and objects.
 - 5) Rich Standard Library: Comes with lots of built-in modules and functions.
- Print ("name: nanna he saru (your.name)!")

Python Basics - 1

1. Variables in Python.

Variables in Python are used to store data values. They are created when you assign a value to them, and you don't need to declare their type (Python is dynamically typed).

Syntax for variable Assignment:

`X = 5` # Assigning an integer value to the variable `x`
`Y = "Hello"` # Assigning a string value to the variable `Y`

Variable naming Rules:

- variable names can contain letters (a-z, A-Z), numbers (0-9), and underscores (_).
- Variable names must start with a letter or an underscore.
- Variable names are case-sensitive (name and Name are different).

Example:

`age = 25`

`name = "John"`

`is_student = True`

2. Data Types in Python

Python has various built-in data types. Some common ones are:

- `int`: For integers (e.g. 1, -3, 100)
- `float`: For floating-point numbers (e.g. 3.14, -0.001)

- str: For String (e.g., "Hello", "Python")
- bool: For boolean values (True or False)

TYPE checking

You can use the `type()` function to check the type of a variable

```
x = 10
```

```
print(type(x)) # output: <class 'int'>
```

3. TYPE conversion

Python allows you to convert between data types using functions like `int()`, `float()`, `str()`, etc.

Ex `x = "10"` # x is a string
 `y = int(x)` # convert string to integer
 `z = float(y)` # convert integer to float
 `print(z)` # output: 10.0

4. Arithmetic operators

Python supports basic arithmetic operations like addition, subtraction, multiplication, division, and more.

Common operators:

- + (Addition)
- - (Subtraction)
- * (multiplication)
- / (Division)
- // (Floor Division)
- % (modulus)
- ** (Exponentiation)

Examples:

a = 10

b = 3

Print (a+b)

Print (a-b)

Print (a*b)

Print (a/b)

Print (a//b)

Print (a % b)

Print (a**b)

5) Assigning values to multiple variables
Python allows you to assign value to multiple variable in a single line.

Example:

x, y, z = 10, 20, 30

Print (x)

Print (y)

Print (z)

You can also assign the same value to multiple variables in one line

x = y = z = 100

Print (x, y, z)

100

6. variable Reassignment

You can change the value of a variable at any point in your Program.

Example:

```
x = 5
```

```
Print (x)
```

```
x = 10
```

```
print (x)
```

Swap

A Python Program can swap the value of two variables efficiently, with a common Pythonic approach using tuple packing / unpacking and a third variable.

1. Swapping with a Third variable

This traditional method uses a temporary variable to hold one of the values during the swap process.

Initialize variables

```
var 1 = 10
```

```
var 2 = 20
```

```
print ("Before swap : var 1 = {var 1} , var 2 = {var 2}")
```

Use a third variable (temp) for swapping

```
temp = var 1
```

```
var 1 = var 2
```

```
var 2 = temp
```

```
Print ("After swap: var 1 = {var 1} , var 2 = {var 2}")
```

2. Swapping without a Third variable

Python provides a very elegant and concise way to swap variables using tuple packing and unpacking.

This is the preferred method in Python

Initialize variables

var 1 = 5

var 2 = 15

Print(f"Before swap : var 1 = {var 1}, var 2 = {var 2}")

Pythonic swap using tuple unpacking

var 1, var 2 = var 2, var 1

Print(f"After swap : var 1 = {var 1}, var 2 = {var 2}")

Alternative method (Mathematical Approach for numbers only)

For numerical values, you can also use arithmetic operation to swap the values without a temporary variable. This method is less common in Python programming but works.

Initialize variables

var 1 = 30

var 2 = 40

Print(f"Before swap : var 1 = {var 1}, var 2 = {var 2}")

Swapping using arithmetic operations

var 1 = var 1 + var 2 # var 1 now holds the sum (70)

var 2 = var 1 - var 2 # var 2 now holds the original var 1 (70 - 30 = 40)

var 1 = var 1 - var 2 # var 1 now holds the original (70 - 40 = 30)

Print(f"After swap : var 1 = {var 1}, var 2 = {var 2}")

Input / Output, String Manipulation, and comments

1). Input and output in Python

(a) Input from the user:

In Python, we use the `input()` function to take input from the user. The data entered by the user is always received as a string, so if you want to use it as a different data type (e.g., integer or float), you'll need to convert it using type conversion functions like `int()` or `float()`.

```
name = input("Enter your name: ")
```

```
age = int(input("Enter your age: ")) # convert input to integer
```

(b) Output to the console:

The `print()` function is used to display output to the console. You can use it to display text, variables, or result of expression.

```
print("Hello, " + name + "! You are " + str(age) + " years old.")
```

You can also use f-string (formatted string literals) for more readable code:

```
print(f"Hello, {name}! You are {age} year old.")
```

2) String manipulation

Strings are sequences of characters. Python provides many useful methods to manipulate strings.

Input / Output, String Manipulation, and comments

1). Input and output in Python

(a) Input from the user:

In Python, we use the `input()` function to take input from the user. The data entered by the user is always received as a string, so if you want to use it as a different data type (e.g., integer or float), you'll need to convert it using type conversion functions like `int()` or `float()`.

```
name = input("Enter your name: ")  
age = int(input("Enter your age: ")) # convert input to integer
```

(b) Output to the console:

The `print()` function is used to display output to the console. You can use it to display text, variables, or result of expression.

```
print("Hello, " + name + "! You are " + str(age) + " years old.")
```

You can also use f-string (formatted string literals) for more readable code:

```
print(f"Hello, {name}! You are {age} year old.")
```

2) String manipulation

Strings are sequences of characters. Python provides many useful methods to manipulate strings.

(a) Common String operations!

Concatenation: joining two or more strings together using the (+) operator

```
first_name = "John"
```

```
last_name = "Doe"
```

```
full_name = first_name + " " + last_name
```

```
Print(full_name) # output: "John Doe"
```

- Repetition Repeating a string multiple times using the (*) operator

```
Greeting = "Hello! " * 3
```

```
Print(Greeting) # output: "Hello! Hello! Hello! "
```

(b) String methods:

- upper(): Converts a string to uppercase
- lower(): Converts a string to lowercase.
- strip(): Removes leading and trailing spaces from a string
- replace(old, new): Replaces a substring with another string

```
message = "Hello, world!"
```

```
Print(message.strip()) # output: "Hello, world!"
```

```
Print(message.upper()) # output: "HELLO, WORLD!"
```

```
print(message.replace("world", "Python")) # output: "Hello, Python!"
```

(c) Accessing string characters:

You can access individual characters in a string using indexing. Python uses zero-based indexing. so the first character has an index of 0.

```
text = "python"
print (text [0]) # output : p
print (text [2]) # output : t
```

you can also use negative indexing to start counting from the end of the string

```
print (text [-1]) # output : n
print (text [-3]) # output : h
```

① Slicing Strings :

you can extract a portion (substring) of a string using slicing

```
text = "python programming"
print (text [0:6]) # output : python (extracts from index 0 to 5)
print (text [:6]) # output : python (same as above)
print (text [7:]) # output : programming (from index 7 to the end)
```

③ Comments in Python

comments are ignored by python interpreter and are used to explain the code or leave note for yourself or others. They do not affect the execution of the program.

- Single-line comments start with # :

```
# This is a single-line comment
```

```
print ("Hello, world!")
```

- Multi-line comments can be written using triple quotes ("""). These are often used to write detailed explanations or temporarily block sections of code:

```
""" This is a multi-line comment.
```

```
It can span multiple lines.
```

```
""" print ("Hello, python!")
```

④ Escape Sequences

Escape Sequences are special characters in string that start with a backslash (\). They are used to represent certain special characters.

Some commonly used escape sequences:

- \n : new line
- \t : Tab space
- \\ : Backslash

Example

```
print("Hello\nworld") # output:
```

```
# Hello
```

```
# world
```

```
print("Hello\tPython") # output: Hello Python
```

Home work

1. Simple Greeting Program: write a Python program that asks the user for their name and age. then prints a personalized greeting message. use both the (+) operator and f-strings for output

Enter your name : Alice

Enter your age : 25

output : Hello, Alice! You are 25 years old.

2. String Manipulation Exercise: write a Python Program that:
- Takes a sentence as input from the user.
 - Prints the sentence in all uppercase and lowercase.
 - Replace all spaces with underscores.
 - Removes leading and trailing whitespace.

Example

Input: " python is awesome! "

Output:

uppercase: "PYTHON IS AWESOME!"

lowercase: "python is awesome!"

replaced: " _python-is-awesome! _"

stripped: "python is awesome."